知简 · 智能文档分析平台

技术文档（参赛作品 · AI协同办公工具赛道）

版本：v1.0

日期：2026 年 10 月 25 日

运行环境：Windows / Node.js ≥ 20 / DeepSeek API

1. 项目概述

1.1 背景

办公与学习场景中，人们经常需要反复翻阅几十页的 PDF 通知、合同、投稿要求或研究资料，只为确认一个数字、一条要求或一个负责人。传统做法是人工翻页 + 关键词搜索（Ctrl+F），但关键词搜索不理解语义，且无法跨文档、跨表格定位；通用大模型虽然能回答问题，但直接提问时它没有读过文档，回答要么靠编造、要么需要用户手动粘贴大段原文。

1.2 目标

本项目实现一个「上传即问」的智能文档分析平台：用户上传 Word / PDF / 文本文件后直接用自然语言提问，系统基于文档内容作答，并且**每条结论都标注来源页码，点击即可跳转查看原文**，让 AI 的回答从「不可验证」变成「可逐条核对」。

1.3 设计原则

- **可信**：只依据上传内容作答，资料不足时明确拒绝，引用编号实时校验、不留死链；
- **轻便**：本地一条命令运行，只需一个大模型密钥，不需要向量数据库、不需要联网检索服务；
- **聚焦**：只做「文档问答 + 引用溯源」一件事，不做摘要、纪要、导出等分散精力的功能。

2. 需求分析

2.1 功能需求

编号	需求	说明
F1	多格式文档解析	支持 PDF、Word（.docx 与旧版 .doc）、纯文本；Word 中的表格、文本框、页眉页脚、脚注批注均需进入索引
F2	自然语言问答	基于已上传文档回答，支持多轮追问（回带最近 6 轮历史）
F3	页码级引用溯源	回答中每个事实性陈述标注来源编号，点击可查看原文所在页并高亮命中词
F4	文档管理	上传（拖拽/多选、进度显示）、列表（页数/片段数/大小）、删除、重复上传去重
F5	流式回答与中断	逐字流式输出，可随时停止生成

2.2 非功能需求

编号	需求	指标
N1	回答可信度	不编造资料外内容；越界引用编号 100% 剔除
N2	部署门槛	单密钥、无数据库、无向量服务；npm install 后一条命令启动
N3	隐私	文档原文只在本机解析与索引；仅检索命中的少量片段随问题发送给模型
N4	性能	流式首字响应；9 页 PDF 解析 < 2 秒
N5	兼容性	桌面浏览器与 390px 移动端无横向溢出；不依赖外部字体/CDN

2.3 典型使用场景

- 赛事/投稿要求核对：上传通知文件，问「最多报几个作品」「图像标识高度要求」；
- 合同要点审查：问「采购金额」「交付期限」「验收标准」并逐条回原文核对；
- 研究资料整理：多份 PDF 一起上传，跨文档提问并定位出处页码。

3. 系统架构设计

3.1 总体架构

系统为单体 B/S 结构，前端与后端同处一个 Next.js 应用，无独立数据库：



3.2 两条核心数据流

入库流：文件 → 文件头嗅探（不信任后缀）→ 选择解析器 → 逐页取文本（带页码）→ 递归切分（段落→句子→硬切，块间重叠 120 字）→ 内存索引 + JSON 落盘。

问答流：问题 → BM25 对全部知识块打分 → 取前 6 块拼成带编号的参考资料 → 先推送 citations 事件（界面立刻显示引用来源）→ 流式调用模型 → 每个 token 经过引用校验器（越界编号剔除）→ 推送 token 事件 → 前端渲染角标按钮。

3.3 接口设计

接口	方法	说明
/api/ingest	POST (multipart)	上传文件入库，返回文档记录；错误带机器可读 code（SCANNED_PDF / ENCRYPTED_PDF / TOO_LARGE 等）
/api/docs	GET / DELETE	文档列表与统计 / 按 id 删除
/api/chat	POST (SSE)	流式问答；事件序列固定为 citations → token* → done（或 error）

3.4 存储设计

索引采用「进程内存 + JSON 落盘」：每次增删后原子写 `.data/index.json`（先写临时文件再 rename），重启自动恢复；写盘失败时降级为纯内存模式不影响使用。单实例上限 8000 块、单文档 600 块，防止极端文档占满内存。

4. AI 技术方案

4.1 模型选型

候选	结论	理由
DeepSeek v4.1-flash	**选用**	中文能力强、上下文 1M、价格低（输入 1~2 元/百万 tokens）、国内直连稳定、支持流式与思考模式切换
本地小模型（4B 级）	不选	长文档问答与引用遵循能力不足，且占用显存
其他商用 API	兼容预留	接口为 OpenAI 兼容格式，改 BASE_URL 即可切换

4.2 检索方案：为什么不用向量数据库

本项目面向「上传几份办公文档、问几个问题」的轻场景。在单文档/少文档语料上实测，本地 BM25 的 Recall@5 已达 100%；引入向量检索需要额外配置 embedding 服务密钥、增加一次网络往返，收益几乎为零。因此检索层采用**零依赖的本地 BM25**：同步纯函数、结果确定可复现、离线可用。这是「成本—效果」权衡后的主动选择，而非能力缺失。

4.3 提示词工程

系统提示词把可信性写成硬约束，而不是依赖模型自觉：

```
1. 只依据【参考资料】回答，不得使用资料之外的知识，不得推测。
2. 资料不足以回答时，直接说「提供的资料中没有相关内容」，不要编造。
3. 每个事实性陈述的句尾必须标注来源编号，格式如 [1]、[2]，可多个如 [1][3]。
4. 引用编号必须来自【参考资料】中真实存在的编号，不得虚构编号。
5. 用简洁的中文回答，先给结论再给依据，不要重复问题。
6. 不确定的地方明确说「资料未明确说明」。
```

参考资料块由检索结果拼装，每条带编号与页码：`[1]（来源：xxx.pdf 第 5 页）`——页码来自解析层而非模型猜测，这是引用可信的前提。

4.4 优化策略

- **无命中短路**：检索 0 命中时不调用模型，直接返回固定话术，零成本、零编造风险；
- **topK=6 + 块重叠 120 字**：控制上下文体积的同时避免答案落在切口上；
- **流式引用校验**：模型输出的 `[n]` 逐片缓冲判定，越界编号实时剔除（处理「[1」被分片拆开的边界）；
- **先推引用后推正文**：用户先看到依据来源，等待体验更好；
- **思考模式可关**：`LLM_CHAT_THINKING=off` 用首字延迟换更低时延；
- **历史截断**：多轮只回带最近 6 轮、单条截断 2000 字，控制 token 成本。

5. 核心功能实现

5.1 文件类型嗅探：不信任后缀

真实环境中遇到过 WPS 把 Word 97-2003 文件存成 `.docx` 后缀（文件头实为 `D0CF11E0` 的 OLE 复合文档）。入口 `sniff.ts` 先读文件头再分流：`%PDF` → PDF 解析器，`PK` → OOXML 解析器，`D0CF11E0` → 旧版 .doc 解析器，其余按文本处理。后缀只用于上传前的快速过滤。

5.2 PDF 解析：坐标重排 + 栏缝检测

真实 PDF 的内容流顺序与视觉顺序无关，一行常被拆成乱序片段。解析器按文本片段的坐标聚类成行（y 容差 3pt）、行内按 x 排序拼接；对双栏版面，用「垂直空白带覆盖率」检测栏缝（gutter），左右栏分行输出，避免两栏文字横向串读。加密 PDF 给出明确错误码 ENCRYPTED_PDF；无文字层的扫描件给出 SCANNED_PDF 并提示先做 OCR。

5.3 Word 解析：自研 OOXML 全量提取

主流库 mammoth 的 `extractRawText` 只取 `w:body` 段落，实测「内容全在文本框」的文档会被判成空文档，表格/页眉页脚/脚注静默丢失。本项目自研 OOXML 提取器：直接解 zip 读 XML，覆盖正文、表格（按行列拼成可读文本）、文本框、页眉页脚、脚注尾注与批注；mammoth 仅作为兜底。旧版 `.doc`（OLE 复合文档）用 word-extractor 直读 WordDocument 流。

5.4 文本切分

递归降级策略：段落 → 句子（按 。！？；切分）→ 固定长度硬切，保证任何输入都能切出块；贪心装箱到目标 700 字，相邻块重叠 120 字，避免答案恰好落在切口。

5.5 BM25 检索

标准 BM25（k1=1.5, b=0.75），对问题分词后对全部块打分，取 score>0 的前 6 块；同分时按文档内原始顺序定序，保证同样输入永远同样输出（可复现、可调试）。分词器同时导出 `queryKeywords` 供前端在原文抽屉里高亮命中词。

5.6 流式引用校验

模型可能编出不存在的编号。校验器维护一个字符缓冲：遇到 `[` 后等待 `]` 凑齐再判定，编号在 1..本次引用总数 内才放行，越界整段剔除并记录；缓冲超过 16 字符强制吐出，避免孤立 `[` 被永远扣住。流结束时 flush 处理残余。前端据此渲染可点角标，绝无死链。

5.7 前端交互

单页两栏：左侧上传（拖拽/多选/进度）与文档列表，右侧问答。回答中的 `[n]` 渲染为角标按钮，点击打开右侧抽屉显示被引用片段原文，命中词标黄；支持停止生成、错误集中提示、未配密钥时的引导横幅。移

动端 390px 单列布局，无横向溢出。

6. 测试与部署

6.1 测试方法

- **端到端（真实模型）**：`tmp/e2e.mjs` 走 HTTP 接口完成「上传→提问→核对答案与页码」，覆盖表格内容、多轮追问、伪装后缀的旧版 .doc、多页 PDF 四类用例；
- **界面实拍**：`tmp/shot-ui.mjs` 用无头浏览器走完整用户路径并抓取页面报错；
- **移动端**：390px 视口检查横向溢出与外部域名请求；
- **服务形态**：生产构建下 `node server.mjs --prod` 验证首页与接口。

6.2 测试结果（2026-10-23，生产构建）

用例	结果
自建 Word（内容在表格里）问金额与负责人	✅ 答 128.5 万元 / 张伟，引用第 1 页
同文档追问合计与质保期	✅ 答 188.5 万元 / 24 个月，引用第 1 页
WPS 旧版 .doc（后缀写成 .docx）	✅ 「最多 5 个作品」「图像 2.8%」「视频 2 秒」全中
9 页 PDF 跨页提问	✅ 首年预算 2.52 万元引第 5 页、47 个馆引第 1/4 页
界面实拍 / 移动端 / 页面报错	✅ 全流程走通，报错 0，高亮 7 处，无横向溢出

6.3 部署方式

```
cd source
npm install
copy .env.example .env.local   # 填 DEEPSEEK_API_KEY（唯一必填）
npm run build && npm start     # 生产模式；或 npm run dev 开发模式
```

索引落盘于 `source/.data/index.json`，重启自动恢复，删除该目录即清库。上传上限 20MB/文件。已知边界：多栏 PDF 按逐行左右拼接取文（面向单栏办公文档）；扫描件（纯图片 PDF）暂不支持，会明确报错提示。

7. 总结与展望

7.1 总结

本项目以「可信、轻便、聚焦」为原则，实现了一个本地一条命令可跑的智能文档分析平台：自研解析内核解决了真实办公文档的三大坑（乱序 PDF、伪装后缀、表格/文本框内容丢失），零向量库的 BM25 检索把部署门槛降到一个密钥，流式引用校验保证每条结论都能点回原文页码。四个真实文档端到端用例全部通过，界面零报错。

7.2 展望

- **扫描件视觉兜底**：无文字层的 PDF 页渲染成图片交给视觉模型转录（已验证 DeepSeek 支持图像理解，每页 ≤1024 tokens），补齐扫描件场景；
- **长文档全文模式**：利用 1M 上下文 + 前缀缓存，小文档整篇送入、按页标记引用，进一步简化链路；
- **图谱检索**：文档量增大后引入轻量图谱 RAG，支持跨段落多跳推理；
- **多会话与导出**：会话历史持久化、问答记录导出 Markdown。

8. 创新点说明

8.1 技术创新点

#	创新点	解决的问题
1	自研 OOXML 全量提取器	主流解析库只取正文段落，表格/文本框/页眉页脚/脚注静默丢失；本方案全量覆盖，mammoth 仅作兜底
2	PDF 坐标重排 + 栏缝检测	PDF 内容流乱序、双栏串读；按坐标聚类成行、垂直空白带检测栏缝
3	文件头嗅探分流	后缀不可信（WPS 旧版 .doc 存成 .docx）；按 D0CF11E0 / PK / %PDF 文件头选解析器
4	流式引用校验	模型编造越界引用编号；逐片缓冲判定、实时剔除，杜绝死链
5	零向量库单密钥架构	轻场景下向量检索收益≈0 却多一个密钥；本地 BM25 离线可用、结果可复现

8.2 应用创新点

#	创新点	价值
1	页码级引用溯源	把「AI 回答」变成「可逐条核对的回答」，页码由解析层给出而非模型猜测
2	显式拒答机制	资料不足时直接说明而非编造，配合无命中短路，从机制上杜绝幻觉
3	一条命令的轻部署	面向个人与小团队的办公文档场景，无需数据库/向量服务/运维，隐私数据不出本机解析层